MPA-MLF, Lab 5 - Feedforward Neural Networks

These exercises are focused on dealing with neural networks. It is strongly recommended to use google collab for these pc labs. Why? Google collab offers a free GPU capacity to train machine learning models. Training NN on GPU can drastically speed up the training process. You can turn on the GPU accelerator in: Runtime -> Change runtime type -> GPU,

The basic terminology

Neural Networks

Think of a neural network as a complex web of neurons (or nodes) that are interconnected to process information, similar to how the human brain works. Each neuron in this network can receive inputs, process them, and pass on the output to other neurons. The goal of a neural network is to transform the inputs (e.g., images, texts, numbers) into a desired output (e.g., identifying what's in an image, translating text, etc.).

title

Layers

A neural network is made up of layers: the input layer (where the data enters the network), one or more hidden layers (where most of the processing happens), and the output layer (where the final prediction is made). You can think of each layer as a filter that increasingly refines the information as it passes through the network. In our case, we will discuss feedforward, fully-connected neural networks. Fully connected means that every neuron in layer n is connected by a weighted connection with all neurons in the layers $n+1$. Feedforward means that data are passed in one direction only, there are no reccurent connections (feedback loops) exist.

title

Neuron

A neuron, or node, is the basic unit of a neural network. It receives input, applies a weight (significance) to that input, adds a bias (a constant value to ensure the neuron activates correctly), and then uses an activation function to determine whether and how strongly to pass its signal to the next layer. Each neuron contributes to the network's ability to make accurate predictions or decisions.|

title

Neuron in the input layer

In the input layer the input neurons are composed of identity only, that means everying that comes to the input comes directly to the output

Training

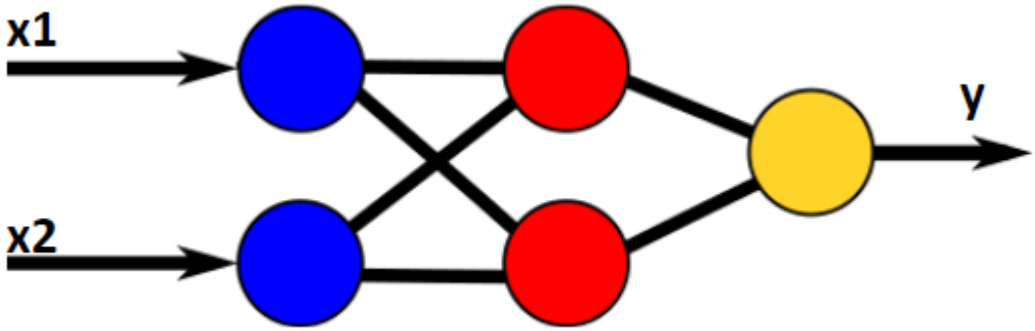
Training a neural network involves feeding it with data for which the correct output is known, allowing the network to adjust its weights and biases to minimize the difference between its predictions and the actual truths. This process is repeated numerous times with many data examples.

Loss Function

A loss function measures how well the neural network is performing, meaning how close or far its predictions are from the actual values. The goal of training is to minimize this loss, or error, making the network's predictions as accurate as possible.

Exercise 1 - XOR problem

During the last lecture, we saw that the single perceptron model with a step function could be used only for solving linearly-separable classification problems. Because of that, a single perceptron can not be trained to be able to behave like an XOR gate. To approximate the XOR gate using a neural network, we need to use the following structure:



The above structure contains three layers: two neurons in the input layer (blue), two in the hidden layer and one in the output layer (yellow). The input layer represents the identity functions (the outputs are equal to the inputs)

We will use *Keras* API to build and train our neural network. Keras is an open-source neural network library written in Python. It is designed to enable fast experimentation with deep neural networks and provides a high-level API for building and training neural networks.

0. First import libraries

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, InputLayer
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np
```

1. prepare data

In this part, the task would be to prepade our dataset, basically it is the logic table for *XOR* function

```
X = [[0, 0], [0, 1], [1, 0], [1, 1]]
y = [0, 1, 1, 0]
```

```
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([0, 1, 1, 0])
```

2. Creating the model

To create a model of NN that is in the picture above, run the following cell

```
model = Sequential()
model.add(InputLayer(shape=(2,)))
model.add(Dense(2, activation='sigmoid'))
model.add(Dense(1, activation='sigmoid'))
```

Sequential is a model type used in Keras for building feedforward neural networks. It is called "sequential" because the layers are stacked sequentially on top of each other, and the output of one layer is passed as input to the next layer.

To create a sequential model, instantiate a Sequential object and then add layers to it using the **add()** method. There are many layers that Keras offers. We will use *Dense layer*

Dense layer represents a fully connected layer, which means that every neuron in the layer is connected to every neuron in the previous layer. In each layer, we will specify an activation function and the number of neurons that layer contains. The input layer is not defined as a dense layer, but instead of that, we will specify the *input_dim* parameter in the layer that follows the input layer. *input_dim* parameter specifies the number of neurons in the input layer

3. Compile the model

To compile yor model run:

```
optimizer = tf.keras.optimizers.SGD(learning_rate=0.1)
model.compile(loss='binary_crossentropy', optimizer=optimizer, metrics
```

keras.complile() is used to configure the learning process before training the model. We need to specify 3 parameters:

- *loss*, *optimizer* and *metrics*
- as our loss function (error function/objective function), we will choose *binary_crossentropy* - loss function often used for binary classification
- *optimizer* is optimization algorithm used for optimizing weights in our training process, we will choose *stochastic gradient descent*
- *metrics* is metric user for model evaluation

4. Model training

In the lecture, we talked about training the Multi-layer perceptron. The training process consists of the following steps:

1. Weight initialization
2. Forward Propagation
3. Compute Loss
4. Backpropagation
5. Update Weights
6. Repeat 2->6 until maximum epochs are reached

When training a machine learning model, the training data is usually divided into batches (parts), and the model updates its weights after processing each batch. The batch size determines how many training examples are included in each batch.

For example, we have 10,000 training examples and a batch size of 100. In this case, the training data would be divided into 100 batches of 100 examples each. During training, the model would process each batch, calculate the loss or error, and update its weights based on the average of the errors in that batch. In other words, if we batch equal to 1. we will update the weights 10 000 times. If we have batch size == 100, we will update the weights only 100 times

To train your model, run the following line of the code:

```
history = model.fit(X, y, epochs=2000, batch_size=1, verbose=0)
```

the method **.fit()**, is used to train the model. We can see that it takes several input parameters:

- *X* - input data
- *y* - label for the input data
- *epochs* number of training epochs (iterations)
- *batch_size* - number of samples in each batch
- *verbose* - set verbose parameter to see the progress of loss and metrics during the training epochs

These are not the only parameters the .fit() method can take. Check official documentation: https://keras.io/api/models/model_training_apis/

We will set the output of our .fit() function to variable *history*, where the loss and other metrics defined above in the .compile() function during the training are saved.

```
Commencez à coder ou à générer avec l'IA.
```

5. Model evaluation

Out training process is finished. To evaluate the performance of our model on test data, tun the following cell of the code:

```
loss, accuracy = model.evaluate(X, y, verbose=0)
print('Accuracy: {:.2f}'.format(accuracy*100))
```

```
Accuracy: 100.00
```

6. Model predictions

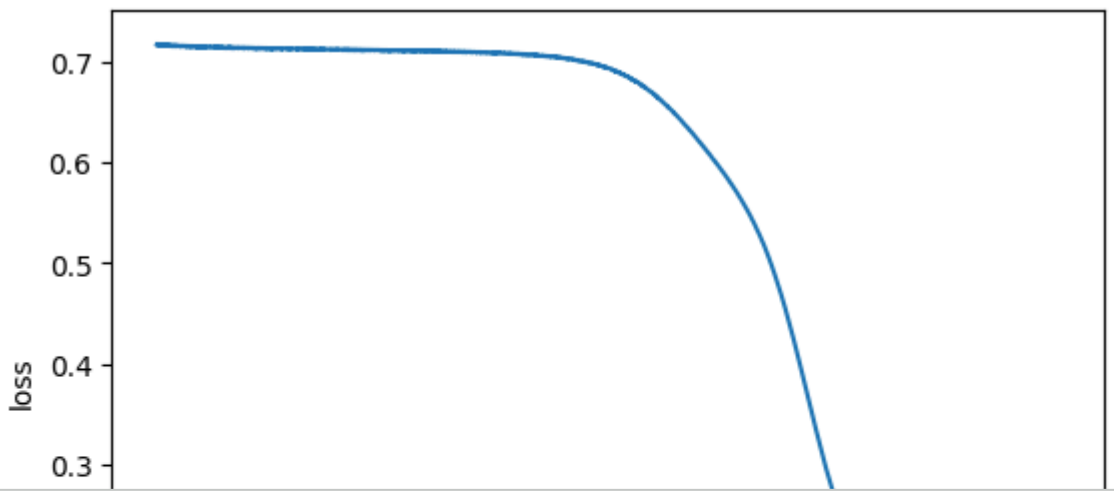
```
for id_x, data_sample in enumerate(X):
  prediction = model.predict(np.array([data_sample]))
  print(f"Data sample is {data_sample}, prediction from model {predict
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 173ms/step
Data sample is [0 0], prediction from model [[0.05698697]], ground_trut
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
Data sample is [0 1], prediction from model [[0.95209414]], ground_trut
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
Data sample is [1 0], prediction from model [[0.9600562]], ground_truth
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step
Data sample is [1 1], prediction from model [[0.04825982]], ground_trut
```

7. Display loss function during the training process and acuracy

```
plt.figure()
plt.plot(history.history['loss'])
plt.xlabel('n epochs')
plt.ylabel('loss')
```

```
Text(0, 0.5, 'loss')
```



TASK:

In this part of the exercise, the task would be to play around with the code above to see the influence of hyperparameters.

As we know from the lecture, neural networks contain two types of parameters:

1. **Learnable parameters** - *weights* and *biases*. these parameters are adjusted in the training process.
2. **Non-learnable parameters (hyperparameters)** - learning_rate, number_of_neurons, number_of_layers, number_of_epochs, type of activation functions in the neurons.., basically any user-defined setting is considered to be hyperparameter

Change the following parameters:

- number of epochs
- learning_rate
- activation functions in layers,
- batch_size,
- verbose,
- number of neurons in the hidden layer

Moreover, see the influence on the training process and results.

Note: Every time we change some hyper-parameters, do not forget to compile the model, to initialize the learnable parameters again

Exercise 2 - Congressional Voting Data

In the attached dataset, results from congressional voting can be found. Your task is to train a model that can recognize that the politician is *republican* or *democrat* based on voting results. We would follow the general machine learning steps that were described in the lecture.

```
import pandas as pd
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import tensorflow as tf
import matplotlib.pyplot as plt
```

1. Loading dataset

First mount your google drive to google colab file.

```
path_to_dataset = 'voting_complete.csv' # change the PATH
pd_dataset = pd.read_csv(path_to_dataset)
```

```
pd_dataset
```

	Unnamed: 0	Class Name	handicapped-infants	water-project-cost-sharing	adoption-of-the-budget-resolution	physician-fee-freeze
0	0	republican	n	y	n	y
1	1	republican	n	y	n	y
2	2	democrat	?	y	y	?
3	3	democrat	n	y	y	n
4	4	democrat	y	y	y	n
...	...	...	...	...	...	...
430	430	republican	n	n	y	y
431	431	democrat	n	n	y	n
432	432	republican	n	?	n	y
433	433	republican	n	n	n	y
434	434	republican	n	y	n	y

435 rows × 18 columns

Étapes suivantes : Générer du code avec pd_dataset New interactive sheet

2. Train/Test Split

Fistlty we need to split our dataset into train and test. We will use 80% of dataset as our trainset and 20% od dataset as our testset. You can use functions included in *keras*, *scikit-learn*, or you can write your own:

```
# define a function for train and test split

def train_test_split(pd_data: pd.DataFrame, test_ratio: float = 0.2) -
    pd_dataset = pd_data.copy()
    pd_dataset = pd_dataset[pd_dataset.columns[1:]]
    index = np.arange(len(pd_dataset))
    index = np.random.permutation(index)
    train_ammount = int(len(index)*test_ratio)
    train_ids = index[train_ammount:]
    test_ids = index[:train_ammount]
```

```
    train_dataset = pd_dataset[pd_dataset.index.isin(train_ids)].reset
    test_dataset = pd_dataset[pd_dataset.index.isin(test_ids)].reset_i

    train_dataset = train_dataset[train_dataset.columns[1:]]
    test_dataset = test_dataset[test_dataset.columns[1:]]

    return train_dataset[train_dataset.columns[1:]], train_dataset[tra
```

```
x_train, y_train, x_test, y_test = train_test_split(pd_dataset)
```

3. Data examination

The task would be to examine the dataset. Check:

1. Is it a classification of regression task?
2. How many data samples do we have?
3. How many features do we have?
4. What data types do we have in our dataset?
5. Are there any missing values?
6. How many labels do we have?

```
x_train
```

	handicapped-infants	water-project-cost-sharing	adoption-of-the-budget-resolution	physician-fee-freeze	el-salvador-aid	religiou groups-i schoc
0	n	y	n	y	y	
1	n	y	n	y	y	
2	?	y	y	?	y	
3	n	y	y	n	?	
4	y	y	y	n	y	
...	...	...	...	...	...	
343	y	n	y	n	?	
344	n	n	y	y	y	
345	n	n	y	n	n	
346	n	?	n	y	y	
347	n	y	n	y	y	

348 rows × 16 columns

Étapes suivantes : Générer du code avec `x_train` New interactive sheet

4. Data preprocessing

When you preprocess your traing data, do not forget that you need to apply the same preprocessing also for your test set. For example: If you decide to delete some columns in your train set, you have to delete the same columns in your test set.

Possible preprocessing steps (try several and see the influence of your preprocessing on your results)

- Replace missing values with any data imputation technique (for example, the most occurring value in the column), then perform one-hot encoding or label encoding of your data
- Consider the missing value to be the third category 'unknown' and then perform one-hot encoding or label encoding

The target value also has to be encoded. This can be done by one-hot encoding or label encoding.

```
def preprocess_features(df: pd.DataFrame, fit_mode: bool = True,
                        mode_values: dict = None) -> tuple:
```

```
    """
    Preprocessing pipeline:
        - Replace '?' (missing values) with the most frequent value ir
        - Label encode: 'n' -> 0, 'y' -> 1
        Returns preprocessed array and the mode_values dict (for reuse
    """
    df = df.copy()

    # Step 1: Replace '?' with column mode (fit only on train)
    if fit_mode:
        mode_values = {}
        for col in df.columns:
            # compute mode ignoring '?'
            col_no_missing = df[col][df[col] != '?']
            mode_values[col] = col_no_missing.mode()[0]

    for col in df.columns:
        df[col] = df[col].replace('?', mode_values[col])

    # Step 2: Label encode 'y' -> 1, 'n' -> 0
    df = df.replace({'y': 1, 'n': 0})

    return df.values.astype(np.float32), mode_values

def preprocess_labels(series: pd.Series) -> np.ndarray:
    """Encode 'republican' -> 0, 'democrat' -> 1"""
    return (series == 'democrat').astype(np.float32).values


# --- Preprocess train set (fit_mode=True learns the mode values)
X_train_processed, mode_values = preprocess_features(x_train, fit_mode
Y_train_processed = preprocess_labels(y_train)

print("\n=== AFTER PREPROCESSING ===")
print(f"X_train shape : {X_train_processed.shape}")
print(f"Y_train shape : {Y_train_processed.shape}")
print(f"Label distribution: {np.bincount(Y_train_processed.astype(int
```

```
=== AFTER PREPROCESSING ===
X_train shape : (348, 16)
Y_train shape : (348,)
Label distribution: [133 215]
/tmp/ipykernel_29547/2588267359.py:23: FutureWarning: Downcasting behav
  df = df.replace({'y': 1, 'n': 0})
```

```
#WRITE YOUR CODE HERE
```

5. Creating the model

1. Create your model using alteast one hidden layer.

hint: do not create too complex models, this is a very simple task, so it would be enought to use just few neurons in the hidden layers

```
# WRITE YOU CODE HERE
model = Sequential([
    InputLayer(shape=(X_train_processed.shape[1],)),   # 16 input feat
    Dense(16, activation='relu'),                       # Hidden layer
    Dense(8,  activation='relu'),                       # Hidden layer
    Dense(1,  activation='sigmoid')                     # Output: bina
])
```

2. Check what *model.summary()* does

```
# WRITE YOU CODE HERE
model.summary()
```

```
Model: "sequential_1"
┏━━━
```

```
        epochs=100,
        batch_size=16,
        validation_split=0.2,    # 20% of train data used for validation
        verbose=1
    )
```

```
Epoch 99/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9712 - loss:
Epoch 100/100
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9784 - loss:
```

7. Model Evaluation

1. First, apply the same preprocessing you did to train set to test set also

```
# WRITE YOUR CODE HERE
X_test_processed, _ = preprocess_fea

Y_test_processed = preprocess_labels
```

```
/tmp/ipykernel_29547/2588267359.py:23: FutureWarning: Downcasting behav
  df = df.replace({'y': 1, 'n': 0})
```

2. Evaluate the model, print final accuracy and loss

```
# WRITE YOUR CODE HERE
test_loss, test_accuracy = model.eva
print(f"\n=== TEST RESULTS ===")
print(f"Test Loss     : {test_loss:.
print(f"Test Accuracy : {test_accura
```

```
=== TEST RESULTS ===
Test Loss     : 0.0742
Test Accuracy : 95.40%
```

3. Plot loss and validation loss depending on the training epochs into one graph. In another graph, plot accuracy and validation accuracy

```
# WRITE YOUR CODE HERE
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history.history['loss'],
plt.plot(history.history['val_loss']
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Loss vs Validation Loss')
plt.legend()
plt.grid(True)

# Plot Accuracy and Validation Accur
plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy']
plt.plot(history.history['val_accura
plt.xlabel('Epochs')
```

```
plt.ylabel('Accuracy')
plt.title('Accuracy vs Validation Ac
```